



PROYECTO FINAL DE LÓGICA COMPUTACIONAL

UNIVERSIDAD ANTONIO NARIÑO

NICOL DAYANA SERNA CASTAÑEDA
KAREN DANIELA ARCOS LÓPEZ
LEIDY DANIELA GALINDO LETRADO

GRUPO #8
PETLANDYA



PETLANDYA

VERSIÓN 1.0

FECHA INICIAL DE ELABORACIÓN:
20 DE MARZO DEL 2025 – 31 DE MAYO DEL
2025

AUTORES:

LEIDY DANIELA GALINDO LETRADO

KAREN DANIELA ARCOS LÓPEZ

NICOL DAYANA SERNA CASTAÑEDA

ENTIDAD:

LÓGICA COMPUTACIONAL –
UNIVERSIDAD ANTONIO NARIÑO

PETLANDYA

- **MANUAL DEL USUARIO**

- ❖ **FUNCIONAMIENTO GENERAL**

Petlandya es un programa que maneja una interfaz por consola, basada en un menú de opciones numeradas donde el usuario puede ingresar datos según corresponda su orden, fue creado principalmente para manejar los datos de una exposición canina, donde se permite ingresar los datos de cada perro participante, el dueño del canino, los jueces y el puntaje que se les otorgara; también permite buscar el perro más viejo, el perro ganador y el perro que presente menor puntaje.

- ❖ **MENU PRINCIPAL Y FUNCIONALIDAD**

- ◆ Ingresar perros: permite ingresar uno o más perros al concurso, se solicitará el nombre del canino, la raza y la edad.
- ◆ Ingresar dueños: permite ingresar los dueños de los perros anteriormente registrados, se solicitará el nombre, identificación, teléfono y correo.
- ◆ Ingresar inscripciones: esta opción relaciona los perros participantes con sus respectivos dueños y solicita ingresar el ID de participación
- ◆ Ingresar exposiciones: guarda los datos del evento referentes al nombre, el lugar, la fecha y la cantidad de perros
- ◆ Ingresar jueces: Permite registrar hasta 3 jueces con nombre, ID y experiencia.
- ◆ Ingresar puntajes: Se ingresan los puntajes que cada juez otorga a cada perro.
- ◆ Mostrar perro ganador: Identifica al perro con mayor promedio de puntaje.
- ◆ Mostrar perro más viejo: Muestra el perro de mayor edad.
- ◆ Buscar información de un perro: Permite localizar un perro por nombre y visualizar sus datos. Si no está registrado, ofrece la opción de hacerlo.
- ◆ Mostrar perro con el menor puntaje: Identifica al perro con el promedio más bajo.
- ◆ Salir: Finaliza la ejecución del sistema.

- ❖ **PRECAUSIONES Y BUENAS PRACTICAS DE USO**

- ◆ Asegúrese de ingresar al menos un perro antes de usar otras funcionalidades.
- ◆ Verifique que los nombres de perros sean únicos para evitar conflictos.
- ◆ Ingrese los jueces antes de asignar puntajes.
- ◆ Siga el orden lógico del menú para evitar datos incompletos (primero perros, luego dueños, inscripciones y jueces).
- ◆ No cierre el programa abruptamente para evitar pérdida de información.
- ◆ Los datos ingresados no se almacenan permanentemente.

❖ PREGUNTAS FRECUENTES

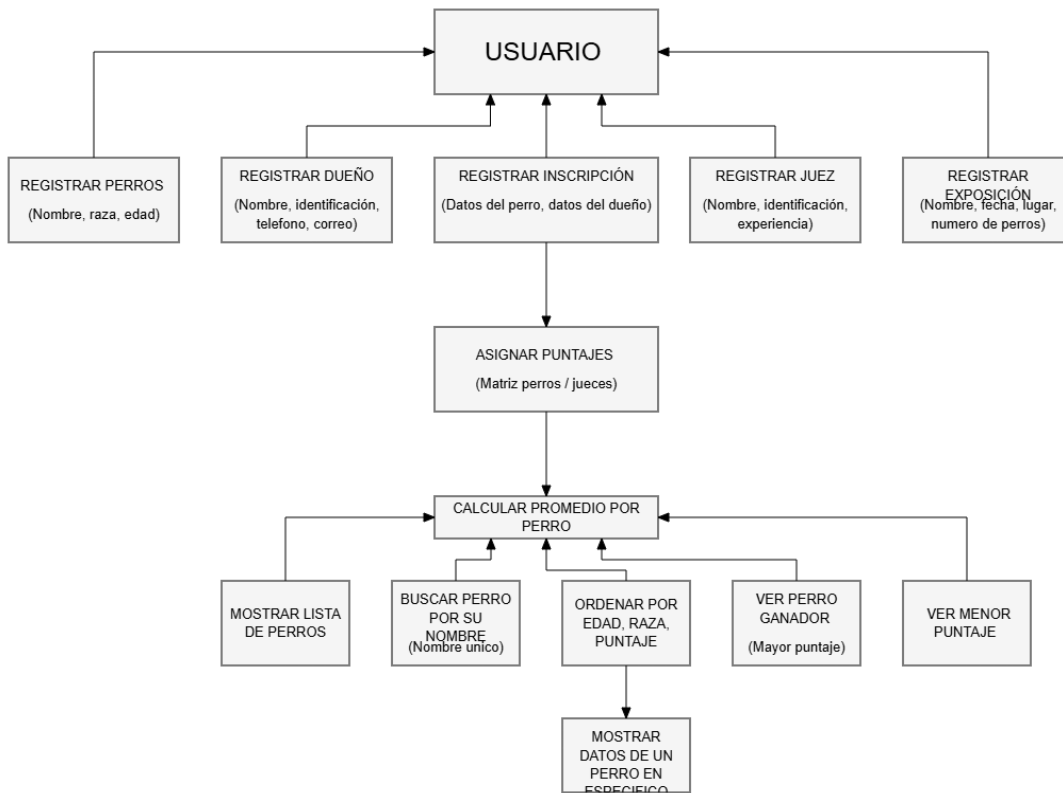
- ◆ ¿Qué ocurre si ingreso un perro que ya existe?
- ◆ ¿Qué ocurre si el juez no otorga ningún puntaje?
- ◆ ¿Puedo cambiar los datos después de ingresarlos?
- ◆ ¿Se pueden ingresar más jueces?

❖ DOCUMENTACIÓN TECNICA

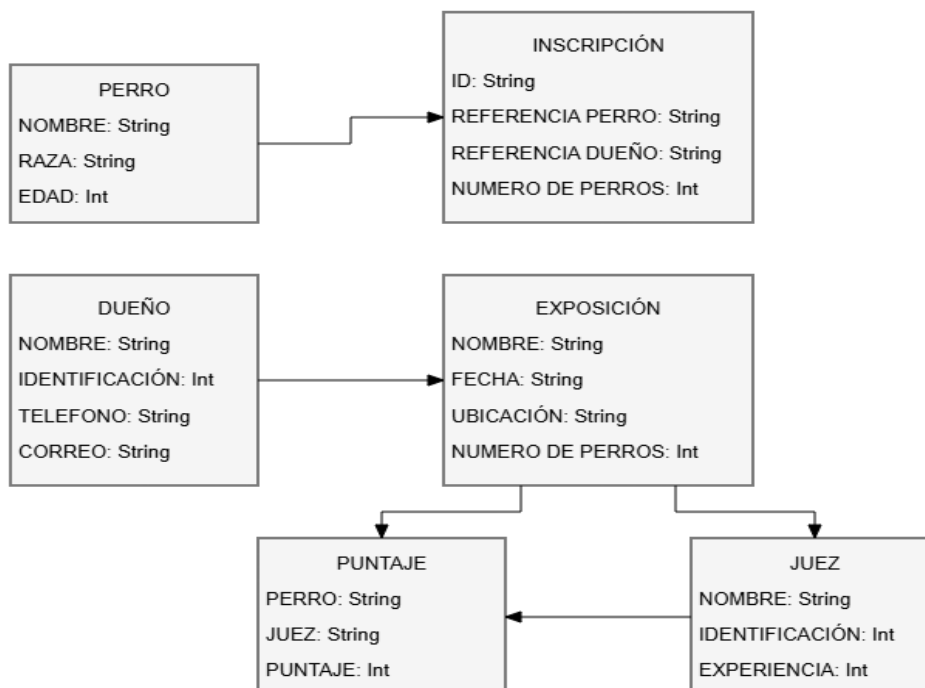
◆ DIAGRAMA ENTIDAD – RELACIÓN



◆ DIAGRAMA DE CASOS DE USO



◆ DIAGRAMA DE CLASES



◆ BITACORA DE CAMBIOS Y NOVEDADES

1. FECHA: 15 de marzo del 2025

DESCRIPCIÓN: En esta primera entrega se implementaron las variables que tenia que mostrar el código, datos de los perros, perro mas viejo, datos de los dueños, datos de la exposición, jueces, etc.

Se entrego por escrito, los requerimientos funcionales que iban a estar dentro del código en entregas futuras.

EVIDENCIA:

```
1  #include <stdio.h>
2  #include <iostream>
3  using namespace std;
4  int main()
5
6  {
7      int opcion;
8      cout<<"digite la opcion a la que quiere acceder";
9      cin>>opcion;
10     switch (opcion){
11     case 1:
12         cout<<"mostrar lista de perros";
13         break;
14         for
15     case 2:
16         cout<<"mostra informacion de un perro en especifico";
17         break;
18     case 3:
19         cout<<"registrar un nuevo perro";
20         break;
21     case 4:
22         cout<<"localizar un perro por su nombre";
23         break;
24     case 5:
25         cout<<"localizar un perro por su nombre";
26         break;
27     case 6:
28         cout<<"mostrar perro ganador";
29         break;
30     case 7:
31         cout<<"perro mas viejo";
32         break;
33     default:
34         cout<<"no esta dentro del rango";
35     }
36 }
```

2. FECHA: 22 de marzo del 2025

DESCRIPCIÓN: Para esta entrega, se realizo el problema soluble que iba a tener nuestro proyecto, que en este caso sería la creación de una aplicación que muestre ciertos datos sobre los perros que participan en un desfile, también en otra tabla la descripción de cada requerimiento y que se obtendría de este.

EVIDENCIA:

Problema:	Crear una aplicación que muestre la lista de los perros registrados en la exposición, ordenado por raza, edad y puntos otorgados.		
Cliente	Organizador de los desfiles		
Usuario	Asistentes al desfile		
Requerimiento funcional	• Raza del perro • Edad del perro • Nombre del perro • Puntos otorgados en el desfile		
Mundo del problema	Se requiere una aplicación que exhiba un inventario exhaustivo de las razas de perros que participan en el desfile, que indique la edad, el nombre y los puntos asignados, que reconozca al animal más viejo, que facilite el registro de un nuevo perro y que muestre al canino con la puntuación más baja, así como permitir la búsqueda de un perro a través de su nombre; al elegir un perro en particular, se posibilite el acceso a toda la información relacionada con él.		
Requerimiento no funcional	• Ofrecer una interfaz más limpia y rápida • Un uso intuitivo y fácil • Funcionamiento veloz • Garantizar la protección de los datos de los perros • La aplicación debe estar disponible el 99% del tiempo, y en caso de no ser así debe tener un mantenimiento eficaz. • Se debe aceptar una infinidad de registros, sin afectar el funcionamiento de la aplicación		
Requerimiento funcional 1	NOMBRE:	Mostrar lista de perros.	
	RESUMEN:	Muestra en la pantalla la lista de perros con sus datos correspondientes.	
	ENTRADAS:	• Opción lista de perros.	
	RESULTADO:	Muestra en la pantalla la lista de perros teniendo en cuenta la opción que seleccione el usuario ya sea la raza, puntos o la edad.	
Requerimiento funcional 2	NOMBRE:	Mostrar los datos relevantes de un perro en específico.	
	RESUMEN:	Muestra los datos relevantes de un perro que sea seleccionado por el usuario.	
	ENTRADAS:	Archivo de la información del perro.	
	RESULTADO:	Muestra la información de un perro que se seleccione, esta incluye: edad, raza, dueño, puntos otorgados, descripción de las características físicas del canino.	

3. FECHA: 29 de marzo del 2025

DESCRIPCIÓN: En esta entrega implementamos en el código los requerimientos funcionales junto con un do while y de manera escrita adjuntamos el análisis de datos, que correspondía a los valores posibles que se podían obtener de cada requerimiento, implementándole un lenguaje c++, de manera que pudiera admitir números únicos o una cadena de caracteres.

EVIDENCIA:

```

2 #include <iostream>
3 using namespace std;
4 int main()
5 {
6     cout<<"Bienvenido a PETLANDYA\n";
7     cout<<"Selecciona una opcion"<<endl;
8     int opcion;
9
10
11 do
12 {
13     cout<<"\n1.lista, 2.informacion, 3.registrar, 4.buscar, 5.ganador, 6.perro mas viejo, 7.saliir";
14     cin>>opcion;
15     switch (opcion){
16     case 1:
17         cout<<"mostrar lista de perros";
18         break;
19     case 2:
20         cout<<"mostrar informacion de un perro en especifico";
21         break;
22     case 3:
23         cout<<"registrar un nuevo perro";
24         break;
25     case 4:
26         cout<<"localizar un perro por su nombre";
27         break;
28     case 5:
29         cout<<"mostrar perro ganador";
30         break;
31     case 6:
32         cout<<"perro mas viejo";
33         break;
34     case 7:"saliir";
35         break;
36
37     default:
38     {
39         cout<<"intentalo de nuevo";
40     }
41 }
42 }while(opcion != 7);
43 cout<<"Gracias por ingresar a PETLANDYA";
44
45
46 }

```

4. FECHA: 26 de abril del 2025

DESCRIPCIÓN: Para esta entrega, se le aplicaron los void a cada requerimiento funcional,

EVIDENCIA:

```

20 void perro::mostrarNombre(){
21     cout<<"El nombre del perro es: \n";
22     cout<<nombre;
23 };
24
25 void perro::asignarRaza(){
26     cout<<"Ingresa raza del perro: \n";
27     cin>>raza;
28 };
29
30 void perro::asignarEdad(){
31     cout<<"La edad del perro es: \n";
32     cout<<edad;
33 };
34 void perro::asignarPuntaje(){
35     cout<<"El puntaje obtenido es: \n";
36     cout<<puntaje;
37 };
38 void perro::mostrarLinkfoto(){
39     cout<<"El link de la foto es: \n";
40     cout<<linkfoto;

```


5. FECHA: 03 de mayo del 2025

DESCRIPCIÓN: Para esta entrega, ya tenemos que implementar los case, que corresponden a cada requerimiento funcional, permitiendo el ingreso de la cantidad de perros que el sistema permitiera.

EVIDENCIA:

```
switch (opcion) {
    case 1:
        for (int i = 0; i < 8; i++) {
            cout << "Ingresar datos del perro #" << i+1 << endl;
            cin >> listaperros[i].nombre;
            cin >> listaperros[i].Raza;
            cin >> listaperros[i].puntaje;
            cin >> listaperros[i].edad;
        }
        break;

    case 2:
        for (int i = 0; i < 8; i++) {
            cout << "Ingresar datos del dueño #" << i+1 << endl;
            cin >> datosdueño[i].nombre;
            cin >> datosdueño[i].identificación;
            cin >> datosdueño[i].telefono;
            cin >> datosdueño[i].correo;
        }
        break;

    case 3:
        for (int i = 0; i < 8; i++) {
            cout << "Ingresar datos de la inscripción #" << i+1 << endl;
            cin >> datosperroydueño[i].iddeinscripcion;
            cin >> datosperroydueño[i].referenciaperro;
            cin >> datosperroydueño[i].referenciadueño;
            cin >> datosperroydueño[i].fechadeinscripcion;
        }
        break;
}
```

6. FECHA: 10 de mayo del 2025

DESCRIPCIÓN: Para esta entrega, teníamos que permitir que el ingreso de datos fuera dinámico, es decir que si quería ingresar x cantidad de perros se lo permitiera y que, si se reiniciaba el sistema, pudiera ingresar una cantidad menor que x o mayor que x.

EVIDENCIA:

```
int main() {
    int x;

    cout << "\n==== Bienvenido a PETLANDYA ==== \n";

    cout << "cuantos perros quieres ingresar";
    cin >> x;
    perro listaperros[x];
    dueño datosdueño[x];
    inscripción datosperroydueño[x];
    raza datosraza[x];
    Exposición datosexposición[1];
    juez datosjuez[3];
    puntaje puntajeperro[x];

    int opcion;
    do {
        cout << "1. Ingresar perros\n";
        cout << "2. Ingresar dueños\n";
        cout << "3. Ingresar inscripciones\n";
        cout << "4. Ingresar razas\n";
        cout << "5. Ingresar exposiciones\n";
        cout << "6. Ingresar jueces\n";
        cout << "7. Ingresar puntajes\n";
        cout << "8. Mostrar datos (no implementado)\n";
        cout << "9. Salir\n";
        cout << "==== \n";
        cout << "Ingrese una opción: ";
        cin >> opcion;
        cin.ignore(); // Limpia buffer
    } while (opcion != 9);
}
```

7. FECHA: 17 de mayo del 2025

DESCRIPCIÓN: Para esta entrega, implementamos una matriz que nos permitiera ver la relación de perros y jueces, visualizando sus puntajes y el promedio de cada perro.

EVIDENCIA:

```
case 8:
    cout << "\nMatriz de puntajes (Perros vs Jueces):\n\n";

    // Encabezado de tabla
    cout << "PERROS\t";
    for (int j = 0; j < 3; j++) {
        cout << datosjuez[j].nombrejuez << "\t";
    }
    cout << endl;

    // Separador visual
    cout << "-----\n";

    // Datos por cada perro
    for (int i = 0; i < x; i++) {
        cout << listaperros[i].nombre << "\t";
        for (int j = 0; j < 3; j++) {
            cout << puntaje_vs_jueces[i][j] << "\t";
        }
        cout << endl;
    }
    break;
```

8. FECHA: 24 de mayo del 2025

DESCRIPCIÓN: Para esta entrega el código ya tenía que estar un 80% funcional, permitiendo que la matriz se viera mas agradable para el usuario, que se pudiera solicitar el perro mas viejo, que si se ingresaba un perro que no estaba registrado, diera la opción de registrarlo o volver al menú inicial, también tenía que estar aplicado que si no se registraban perros, no permitiera al usuario avanzar a otra opción del menú sin haber ingresado la primera y que permitiera visualizar la información de un perro en específico.

EVIDENCIA:

```

void mostrarGanador(perro listaperros[], int puntaje[][3], int totalPerros) {
    float mayor_promedio = -1;
    string nombre_ganador = "";
    for (int i = 0; i < totalPerros; i++) {
        float suma = 0;
        for (int j = 0; j < 3; j++) {
            suma += puntaje[i][j];
        }
        float promedio = suma / 3.0;
        if (promedio > mayor_promedio) {
            mayor_promedio = promedio;
            nombre_ganador = listaperros[i].nombre;
        }
    }
    cout << "El perro ganador es: " << nombre_ganador << " con un promedio de " << mayor_promedio << endl;
}

void mostrarPerroMasViejo(perro listaperros[], int totalPerros) {
    if (totalPerros == 0) {
        cout << "No hay perros registrados.\n";
        return;
    }

    int mayor_edad = listaperros[0].edad;
    string nombre_perro_mayor = listaperros[0].nombre;

    for (int i = 1; i < totalPerros; i++) {
        if (listaperros[i].edad > mayor_edad) {
            mayor_edad = listaperros[i].edad;
            nombre_perro_mayor = listaperros[i].nombre;
        }
    }

    cout << "El perro más viejo es: " << nombre_perro_mayor << " con " << mayor_edad << " años\n";
}

void mostrarInformacionPerro(perro listaperros[], int& totalPerros) {
    string nombre_buscar;
    bool encontrado = false;
    cout << "Ingrese el nombre del perro a buscar: ";
    getline(cin, nombre_buscar);
}

```

9. FECHA: 31 de mayo del 2025

DESCRIPCIÓN: Ya para esta fecha el código ya está completamente terminado, con los parámetros requeridos en el proyecto #19, también incluía que antes de mostrar el menú, mostrara los nombres de los autores.

EVIDENCIA:

```

int main() {
    cout << "===== BIENVENIDO A PETLANDYA =====\n";
    cout << "AUTORES:\n";
    cout << "Leidy Daniela Galindo Letrado\n";
    cout << "Nicol Dayana Serna Castañeda\n";
    cout << "Karen Daniela Arcos López\n";

    int eleccion;
    cout << "\n¿Deseas ingresar al menú principal?\n1. Sí\n2. No\nIngrese opción: ";
    cin >> eleccion;
    cin.ignore();

    if (eleccion == 1) {
        mostrarMenu();
    } else {
        cout << "Programa finalizado.\n";
    }
}

```

♦ CONOCIMIENTOS PREVIOS

- ❖ **Conocimiento básico de la interfaz de usuario:** Saber cómo navegar, tocar, deslizar y usar los botones en la pantalla.
- ❖ **Capacidad de lectura e interpretación:** Entender las instrucciones y mensajes que la aplicación muestra.
- ❖ **Comprender las funciones básicas:** Saber cómo realizar las tareas principales de la aplicación, como buscar, registrarse, encontrar resultados, etc.
- ❖ **Conocimiento de la personalización:** Saber cómo personalizar la aplicación según las preferencias personales.
- ❖ **Conocimiento de las actualizaciones:** Entender la importancia de mantener la aplicación actualizada

♦ REQUISITOS TECNICOS PREVIOS

- ❖ Conexión a internet estable, esto facilita que la ejecución del código sea más rápida a la hora de usar cada requerimiento.
- ❖ Compatibilidad con dispositivos fijos o estáticos, esto permite que el código pueda ser utilizado en cualquier dispositivo y se acomode a los diferentes contextos de los usuarios.
- ❖ Disponible en motores de búsqueda, debido a que puede ser usado en un compilador de programación permite que no solo se limite a un motor de búsqueda si no que el usuario tenga la libertad de elegir el de su preferencia.
- ❖ Enfocada para países hispanohablantes.

♦ CAPACIDADES TECNICAS MINIMAS DEL EQUIPO

- ❖ Habilidades en programación en sus diferentes lenguajes tales como c++, necesarios para el adecuado desarrollo del código que se presenta.
- ❖ Comprender a profundidad como se llevan a cabo los diferentes procesos necesarios para el uso adecuado del código y la ejecución de cada requerimiento funcional.
- ❖ Adaptar adecuadamente las necesidades del público para implementarlas de forma correcta y eficiente.
- ❖ Entender como desde diferentes contextos se pueden aplicar los contenidos y conceptos vistos a lo largo del desarrollo del código.

♦ SOFTWARE ASOCIADO NECESARIO

- ❖ Es necesario ejecutar el código desde diferentes softwares compilables (sistemas legibles para el lenguaje c++) como: GDB online Debugger, Programiz, entre otros.

♦ MECANISMOS PARA ACCEDER AL SISTEMA

- ❖ El usuario debe acceder a un navegador de su preferencia y buscar un software compilador compatible con su dispositivo para el lenguaje c++, este puede ser online o puede descargar algún programa en específico como Programiz.
- ❖ Al seleccionar el compilador puede ingresar al link que se comparte o copiar el código del mismo, esto le permitirá acceder a todas las funciones que se presentan, no olvide seguir la guía de usuario para obtener una mejor experiencia.

♦ DATOS DE CONTACTO DE LOS AUTORES

1. NOMBRE: Leidy Daniela Galindo Letrado
CORREO INSTITUCIONAL: lgalindo364@uan.edu.co
2. NOMBRE: Karen Daniela Arcos López
CORREO INSTITUCIONAL: karcos696@uan.edu.co
3. NOMBRE: Nicol Dayana Serna Castañeda
CORREO INSTITUCIONAL: nserna898@uan.edu.co